

# SIMATS ENGINEERING

## ASSESSMENT TOOL 3

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

---

### COURSE OUTCOME COVERED

**CO5:** Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting  
(BL 3)

Assessment Tool Weightage: 10%

---

QUESTIONS: 10

Annexure A

Total Marks:20

### MOCK INTERVIEW QUESTIONS

---

#### Code of Conduct:

I Chalamala Sai Harshitha (192424346) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.

#### Signature of the student:

C. Sai Harshitha

## Annexure A

- 1. You are working as a data analyst in a hospital where patient data is continuously collected and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.**

### **1. Reading the CSV Data**

The patient data can be loaded into a Pandas DataFrame using:

```
import pandas as pd
import numpy as np
df = pd.read_csv("patient_data.csv")
```

The DataFrame provides an efficient structure for processing tabular patient information.

### **2. Handling Missing Values**

Missing values can be identified using:

```
print(df.isnull().sum())
```

Appropriate techniques can then be applied depending on the column:

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Temperature"] = df["Temperature"].fillna(df["Temperature"].mean())
df = df.dropna(subset=["Patient_ID"])
```

Important fields such as Patient\_ID should not contain missing values.

### **3. Removing Duplicate Entries**

Duplicate records can be removed using:

```
df = df.drop_duplicates()
```

This prevents the same patient information from being counted more than once during analysis and reporting.

### **4. Standardizing Inconsistent Units**

Different units should be converted into a common standard. For example, Fahrenheit temperature values can be converted to Celsius using NumPy:

```
df["Temperature_C"] = np.where(
    df["Unit"] == "F",
    (df["Temperature"] - 32) * 5 / 9,
    df["Temperature"])

```

This ensures consistency during numerical analysis.

## 5. Filtering Data

Pandas filtering can be used to select relevant patient records. For example:

```
high_risk = df[(df["Age"] > 60) & (df["Temperature_C"] > 38)]
```

This extracts records satisfying the specified conditions.

## 6. Indexing

Patient\_ID can be used as an index when it uniquely identifies each patient:

```
df = df.set_index("Patient_ID")
```

Indexing allows patient records to be accessed efficiently.

## 7. Aggregation

The groupby() and agg() functions can be used to generate summary reports:

```
summary = df.groupby("Department")["Temperature_C"].agg(  
    ["mean", "min", "max"])
```

This provides department-wise minimum, maximum, and average temperature values.

## 8. Handling Data Anomalies

Unexpected or invalid values should be identified and handled before generating reports.

For example:

```
df = df[(df["Age"] >= 0) & (df["Age"] <= 120)]
```

```
df = df[(df["Temperature_C"] >= 30) & (df["Temperature_C"] <= 45)]
```

Invalid records can be flagged, logged, corrected where possible, or excluded from analysis.

Unexpected file formats, corrupted records, and invalid data types should also be handled through validation and error-handling mechanisms.

## 9. Saving the Processed Data

After cleaning and standardization, the processed data can be saved as a CSV file:

```
df.to_csv("processed_patient_data.csv", index=True)
```

The resulting file can be used for reporting and further analysis.

## 10. Near-Real-Time Processing

Since patient data is continuously collected, newly generated CSV files can be periodically processed through the same pipeline. Each batch can undergo cleaning, duplicate removal, unit standardization, filtering, validation, and aggregation before being added to the reporting dataset.

## Conclusion

NumPy and Pandas together provide an efficient solution for processing hospital patient data. Pandas handles DataFrame operations, file I/O, indexing, filtering, grouping, aggregation, and data cleaning, while NumPy supports efficient numerical calculations.

2. A financial company provides you with millions of transaction records that must be analyzed to detect unusual spending patterns. Due to system limitations, memory and computation time are critical constraints. Describe how you would use NumPy arrays and Pandas DataFrames to perform filtering, grouping, and statistical analysis efficiently. Explain how you would optimize indexing and aggregation operations, and how you would debug performance bottlenecks or incorrect outputs.

### 1. Loading the Transaction Data

Transaction records can be loaded into a Pandas DataFrame:

```
import pandas as pd
import numpy as np
df = pd.read_csv("transactions.csv")
```

Only the required columns should be loaded when dealing with millions of records:

```
df = pd.read_csv(
    "transactions.csv",
    usecols=["Transaction_ID", "Customer_ID", "Amount", "Category"])
```

This reduces memory consumption.

### 2. Using Appropriate Data Types

Unnecessary memory usage can be reduced by selecting suitable data types:

```
df["Customer_ID"] = df["Customer_ID"].astype("category")
df["Category"] = df["Category"].astype("category")
df["Amount"] = df["Amount"].astype("float32")
```

Using smaller or categorical data types can significantly reduce the memory required for large datasets.

### 3. Efficient Filtering

Pandas can filter transactions based on specified conditions:

```
large_transactions = df[df["Amount"] > 50000]
```

Multiple conditions can also be applied:

```
unusual = df[(df["Amount"] > 50000) & (df["Category"] == "Online")]
```

Vectorized operations are preferred because they process complete arrays or columns efficiently without unnecessary Python loops.

### 4. Grouping Transactions

Transactions can be grouped by customer to identify spending patterns:

```
customer_spending = df.groupby("Customer_ID")["Amount"].sum()
```

Multiple statistical measures can be calculated:

```
summary = df.groupby("Customer_ID")["Amount"].agg(
    ["count", "mean", "sum", "max"])
```

This helps identify customers with unusually high transaction amounts or frequencies.

## 5. Statistical Analysis Using NumPy

NumPy can be used for efficient numerical calculations:

```
mean = np.mean(df["Amount"])
std = np.std(df["Amount"])
```

Transactions significantly higher than the normal spending level can be identified:

```
unusual = df[df["Amount"] > mean + 3 * std]
```

Such statistical analysis can help detect potentially unusual spending patterns.

## 6. Optimizing Indexing

Frequently accessed columns can be used as indexes:

```
df = df.set_index("Customer_ID")
```

An index can improve repeated lookup operations, especially when searching for transactions belonging to a particular customer.

However, indexes should be created only when they provide a performance benefit because maintaining indexes also requires memory.

## 7. Optimizing Aggregation

Vectorized Pandas functions such as `sum()`, `mean()`, `count()`, and `groupby()` should be preferred over explicit loops.

For example:

```
result = df.groupby("Customer_ID")["Amount"].sum()
```

This is generally more efficient than manually iterating through millions of transaction records.

## 8. Handling Large Datasets

When the complete CSV file cannot comfortably fit into memory, the file can be processed in chunks:

```
for chunk in pd.read_csv("transactions.csv", chunksize=100000):
    result = chunk.groupby("Customer_ID")["Amount"].sum()
```

Chunk-based processing reduces peak memory usage and is suitable for very large transaction files.

## 9. Debugging Performance Bottlenecks

Performance can be checked by examining:

- Memory consumption, Execution time
- Expensive filtering operations, Unnecessary DataFrame copies
- Inefficient loops

- Excessive or unnecessary indexes
- Repeated aggregation operations

Operations should be measured and optimized rather than assuming that a particular operation is the bottleneck.

## 10. Debugging Incorrect Outputs

Incorrect results can be investigated by checking:

```
print(df.info())
```

```
print(df.isnull().sum())
```

```
print(df.describe())
```

```
print(df.head())
```

These checks help identify incorrect data types, missing values, abnormal numerical values, and unexpected records.

Duplicate transactions should also be checked:

```
print(df.duplicated().sum())
```

## Conclusion

NumPy and Pandas provide efficient tools for analyzing millions of financial transactions. Memory usage can be reduced through appropriate data types, selective column loading, and chunk-based processing. Vectorized filtering, grouping, aggregation, and statistical operations improve computational efficiency.

- 3. You are given multiple datasets from different departments (sales, marketing, and customer support), each stored in different file formats. Your task is to combine these datasets into a unified DataFrame while ensuring no loss of critical information and maintaining consistent indexing. Explain your approach using Pandas for file I/O, combining, grouping, and sorting. How would you validate the merged dataset and resolve conflicts such as missing or duplicate keys?**

### 1. Reading the Datasets

Pandas provides different functions for reading datasets based on their file formats.

```
import pandas as pd
```

```
sales = pd.read_csv("sales.csv")
```

```
marketing = pd.read_excel("marketing.xlsx")
```

```
support = pd.read_json("support.json")
```

Each dataset is loaded into a separate DataFrame.

### 2. Checking the Structure

Before combining the datasets, their structure should be examined.

```
print(sales.info())
print(marketing.info())
print(support.info())
```

Column names, data types, missing values, and key fields should be checked to ensure compatibility.

### 3. Maintaining Consistent Keys

A common key such as Customer\_ID should be used to connect the datasets.

```
sales["Customer_ID"] = sales["Customer_ID"].astype(str)
marketing["Customer_ID"] = marketing["Customer_ID"].astype(str)
support["Customer_ID"] = support["Customer_ID"].astype(str)
```

Converting the key columns to the same data type prevents incorrect matches during merging.

### 4. Handling Duplicate Keys

Duplicate customer keys should be identified:

```
print(sales["Customer_ID"].duplicated().sum())
```

If duplicate records are caused by repeated entries, they can be removed:

```
sales = sales.drop_duplicates()
```

However, legitimate multiple transactions or support interactions should not be removed simply because the customer ID is repeated.

### 5. Combining the Datasets

The datasets can be merged using the common customer key:

```
merged = pd.merge(
    sales,
    marketing,
    on="Customer_ID",
    how="outer"
)
merged = pd.merge(
    merged,
    support,
    on="Customer_ID",
    how="outer"
)
```

An **outer merge** helps retain records from all departments, thereby reducing the risk of losing critical information.

### 6. Handling Missing Keys

Missing customer IDs can be identified using:

```
print(merged["Customer_ID"].isnull().sum())
```

Records with missing keys should be investigated. If the key cannot be recovered, such records can be flagged separately rather than being incorrectly matched with another customer.

## **7. Handling Missing Values**

After merging, missing values may appear because a customer exists in one department's dataset but not another.

```
print(merged.isnull().sum())
```

Appropriate handling methods such as filling, retaining as missing, or flagging the values can be applied depending on the business requirement.

## **8. Grouping the Unified Data**

The combined DataFrame can be grouped to generate customer-level summaries:

```
summary = merged.groupby("Customer_ID").size()
```

For example, sales information can be aggregated:

```
sales_summary = merged.groupby("Customer_ID")["Sales_Amount"].sum()
```

## **9. Sorting the Data**

The unified dataset can be sorted according to relevant fields:

```
merged = merged.sort_values("Customer_ID")
```

Sorting makes the final dataset easier to inspect and analyze.

## **10. Validating the Merged Dataset**

The merged dataset should be validated by checking:

- Number of records before and after merging
- Missing customer IDs, Duplicate records
- Data types, Missing values
- Unexpected changes in important columns

For example:

```
print(merged.shape)
```

```
print(merged["Customer_ID"].isnull().sum())
```

```
print(merged.duplicated().sum())
```

```
print(merged.info())
```

## **Conclusion**

Pandas provides an efficient approach for combining datasets stored in different formats. Consistent key fields, appropriate merge operations, duplicate handling, missing-value treatment, grouping, and sorting help create a reliable unified DataFrame.



4. A company requires an automated system that reads daily data files, processes them, and generates summary reports along with visualizations. The pipeline must filter relevant data, compute statistics, and produce sorted outputs for dashboards. Describe how you would design this workflow using NumPy and Pandas, including file handling, filtering, aggregation, and plotting. Also explain how you would handle errors such as corrupted files, inconsistent formats, or missing columns.

### 1. Reading Daily Data Files

Pandas can be used to read the daily CSV files:

```
import pandas as pd
import numpy as np
df = pd.read_csv("daily_data.csv")
```

The data is stored in a Pandas DataFrame for further processing.

### 2. Checking and Validating the Data

The structure of the dataset should be checked before processing:

```
print(df.info())
print(df.isnull().sum())
```

The required columns should also be verified:

```
required = ["Customer_ID", "Amount", "Category"]
missing = [col for col in required if col not in df.columns]
```

This prevents the pipeline from processing incomplete datasets.

### 3. Filtering Relevant Data

Pandas filtering can be used to select the required records:

```
filtered = df[df["Amount"] > 1000]
```

Only relevant records are passed to the subsequent processing stages.

### 4. Computing Statistics

Statistical measures can be calculated using Pandas and NumPy:

```
average = np.mean(filtered["Amount"])
maximum = np.max(filtered["Amount"])
minimum = np.min(filtered["Amount"])
```

These values can be used in the summary report.

### 5. Aggregation

The data can be grouped according to a relevant category:

```
summary = filtered.groupby("Category")["Amount"].agg(
    ["count", "sum", "mean"]
)
```

This generates category-wise statistics that can be used for dashboard reporting.

## **6. Sorting the Output**

The summary can be sorted according to the required metric:

```
summary = summary.sort_values("sum", ascending=False)
```

This places the highest-value categories at the top of the dashboard output.

## **7. Generating Visualizations**

The processed data can be visualized using Pandas plotting:

```
summary["sum"].plot(kind="bar")
```

The visualization can provide a quick representation of the summarized data for dashboard users.

## **8. Saving the Processed Data**

The final processed output can be saved for reporting:

```
summary.to_csv("daily_summary.csv")
```

This creates a structured file containing the daily summary.

## **9. Handling Corrupted Files**

File-reading operations should be placed inside error-handling mechanisms:

try:

```
df = pd.read_csv("daily_data.csv")
```

except Exception as e:

```
print("Error while reading file:", e)
```

Corrupted or unreadable files can be logged and excluded from the processing pipeline rather than stopping the entire workflow.

## **10. Handling Inconsistent Formats and Missing Columns**

Different files may contain inconsistent column names or formats. The required structure should therefore be checked before processing:

```
required = ["Customer_ID", "Amount", "Category"]
```

```
if not all(col in df.columns for col in required):
```

```
    print("Required columns are missing")
```

Data types and formats should also be standardized before filtering and aggregation.

## **Conclusion**

NumPy and Pandas can be combined to create an automated daily data-processing workflow. Pandas handles file I/O, filtering, aggregation, sorting, and plotting, while NumPy supports numerical and statistical calculations. Validation and error-handling mechanisms ensure that corrupted files, inconsistent formats, and missing columns do not produce unreliable dashboard results.

5. You are working on an e-commerce platform where customer behavior data is collected in real time. The dataset contains clickstream data with missing values and inconsistent formats. Explain how you would preprocess and structure this data using Pandas. Discuss filtering, feature extraction, indexing strategies, and how you would prepare the data for downstream machine learning tasks.

### 1. Loading the Clickstream Data

The clickstream data can be loaded into a Pandas DataFrame:

```
import pandas as pd
import numpy as np
df = pd.read_csv("clickstream.csv")
```

The DataFrame provides a structured format for processing customer activity records.

### 2. Identifying Missing Values

Missing values can be identified using:

```
print(df.isnull().sum())
```

Appropriate methods can then be applied based on the type of data:

```
df["Product_ID"] = df["Product_ID"].fillna("Unknown")
df["Duration"] = df["Duration"].fillna(df["Duration"].median())
```

Important records with missing essential fields can be removed or flagged for further processing.

### 3. Handling Inconsistent Formats

Data formats should be standardized before analysis. For example, timestamps can be converted into a consistent datetime format:

```
df["Timestamp"] = pd.to_datetime(df["Timestamp"], errors="coerce")
```

Invalid timestamps become missing values and can subsequently be handled appropriately.

### 4. Filtering the Data

Irrelevant or invalid records can be removed using filtering:

```
df = df[df["Duration"] >= 0]
```

Specific customer activities can also be selected:

```
purchases = df[df["Event"] == "Purchase"]
```

This ensures that only relevant and valid records are used for further processing.

### 5. Feature Extraction

Useful features can be extracted from the clickstream data. For example, date and time information can be extracted from the timestamp:

```
df["Hour"] = df["Timestamp"].dt.hour
df["Day"] = df["Timestamp"].dt.day
```

Customer-level behavioral features can also be generated:

```
features = df.groupby("Customer_ID").agg(  
    Total_Events=("Event", "count"),  
    Total_Duration=("Duration", "sum")  
)
```

These features can represent customer activity and engagement.

## **6. Indexing Strategies**

Customer\_ID can be used as an index for efficient customer-level operations:

```
df = df.set_index("Customer_ID")
```

A datetime index can also be useful when analyzing customer behavior over time:

```
df = df.set_index("Timestamp")
```

The choice of index depends on the type of analysis required.

## **7. Structuring the Data**

After preprocessing, the data should be organized into meaningful columns containing customer information, event details, timestamps, product information, and extracted behavioral features.

For example:

```
features = df.groupby("Customer_ID")["Duration"].agg(  
    ["count", "mean", "sum"]  
)
```

This creates a structured customer-level feature dataset.

## **8. Preparing Data for Machine Learning**

The processed features can be separated into input variables for downstream machine learning:

```
X = features[["count", "mean", "sum"]]
```

Missing, invalid, or inconsistent values should be handled before supplying the features to a machine learning model.

## **Conclusion**

Pandas provides an effective approach for preprocessing and structuring e-commerce clickstream data. Missing values and inconsistent formats can be handled during preprocessing, while filtering removes irrelevant records. Feature extraction converts raw customer activity into useful behavioral attributes, and appropriate indexing supports efficient data access. The resulting structured features can then be prepared for downstream machine learning tasks.

## RUBRICS FOR CALCULATION

| Criteria                          | Weightage | Level 4 – Exemplary (5 Marks)                                | Level 3 – Proficient (4 Marks)         | Level 2 – Developing (2–3 Marks)             | Level 1 – Beginning (0–1 Mark)  |
|-----------------------------------|-----------|--------------------------------------------------------------|----------------------------------------|----------------------------------------------|---------------------------------|
| <b>Conceptual Understanding</b>   | <b>30</b> | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps     | Basic understanding with some misconceptions | Poor or incorrect understanding |
| <b>Accuracy of Answer</b>         | <b>20</b> | Completely correct answer with no errors                     | Mostly correct with minor errors       | Partially correct with noticeable errors     | Incorrect or irrelevant answer  |
| <b>Application of Knowledge</b>   | <b>25</b> | Correctly applies concepts to solve the question/problem     | Applies concepts with minor mistakes   | Limited or partially correct application     | No or incorrect application     |
| <b>Completeness of Response</b>   | <b>15</b> | Covers all required parts of the question                    | Covers most parts with minor omissions | Covers only some parts                       | Incomplete or missing response  |
| <b>Clarity &amp; Presentation</b> | <b>10</b> | Clear, concise, and well-organized answer                    | Understandable with minor issues       | Somewhat unclear or poorly structured        | Difficult to understand         |